

AirSense Projesi Mevcut Durum Analizi

(Technical Audit)

1. Proje Amacı ve Mevcut Durum (Target vs. Reality)

- **İlk Hedef:** Kapalı bir odanın hava kalitesini, sıcaklığını analiz etmek ve tehlikeli gaz (yangın/kaçak) durumunda uyarı vermek.
- **Şu Anki Durum:** Sistem **MVP (Minimum Viable Product)** seviyesinde çalışıyor.
 - Sıcaklık ve Nem ölçülüyor.
 - Gaz yoğunluğu (analog veri olarak) ölçülüyor.
 - Tehlike anında **yerel uyarı** (Buzzer) veriliyor.
 - Tüm veriler buluta (Supabase) kaydediliyor ve mobil uygulamadan anlık izlenebiliyor.
 - *Eksik:* Henüz hassas ppm kalibrasyonu yapılmadı, ham veri üzerinden eşik değerle çalışıyoruz.

2. Donanım Katmanı (Hardware Layer) - "Uç Birim"

Burada **Embedded C++ (Arduino Framework)** kullanıyoruz. PlatformIO ile kodluyoruz.

- **Mikrokontrolcü: ESP32 Dev Module (30 Pin).**
 - *Görevi:* Sensörleri okumak, veriyi işlemek, Wi-Fi'a bağlanıp JSON paketi oluşturmak ve sunucuya POST etmek.
- **Sensörler ve Aktüatörler:**
 - **DHT11 (Dijital):**
 - *Bağlantı:* GPIO 4.
 - *İşlevi:* Ortam sıcaklığı ($\pm 2^{\circ}\text{C}$ hata payı) ve nemini okuyor.
 - **MQ-9 / MQ-135 (Analog):**
 - *Bağlantı:* GPIO 34 (ADC1 Kanalı - Wi-Fi ile çakışmaz).
 - *İşlevi:* Ortamdaki yanıcı gaz veya kirli hava yoğunluğunu 0-4095 arasında bir voltaj değeri olarak okuyor. Şu an kodda "Genel Gaz Değeri" olarak işleniyor.
 - **Buzzer (Dijital Çıkış):**
 - *Bağlantı:* GPIO 13.
 - *İşlevi:* Yazılımsal bir eşik değeri (gaz > 1200) aşıldığında tetiklenip sesli uyarı veriyor.
- **Kritik Yazılım Müdahaleleri:**

- **Auto-Reconnect:** Wi-Fi kopsa bile cihaz reset atmadan tekrar bağlanmaya çalışıyor.
- **HTTP Timeout (10s):** Sunucu cevap vermekte gecikirse cihazın kilitletlenmesini (Watchdog Reset) önlemek için bekleme süresini artırdık.

3. Backend Katmanı (Server Side) - "Beyin"

Burada **Python** ve **FastAPI** kullanıyoruz. Bilgisayarında (Localhost) çalışıyor.

- **Framework: FastAPI.**
 - *Neden Seçtik?* Çok hızlı (Asenkron), Swagger (Docs) arayüzünü otomatik veriyor ve IoT için hafif.
- **Veritabanı: Supabase (PostgreSQL).**
 - *Yapı:* Bulutta tutuluyor. Tablolarımızda temperature, humidity, mq9_value, is_alert gibi sütunlar var.
- **API Endpoint'leri:**
 - POST /api/v1/data: ESP32'den gelen ham veriyi alır, doğrular ve veritabanına yazar.
 - GET /api/v1/history/{device_id}: Mobil uygulamanın verileri çektiği yer. Son ölçümleri listeler.
- **Ağ Yapılandırması (Network Config):**
 - Sunucu 0.0.0.0 IP'sine bind edildi (Tüm ağdan gelen isteklere açık).
 - **Windows Firewall Kuralı:** 8000 numaralı port dışarıdan (ESP32 ve Telefondan) gelen trafiğe açıldı.

4. Mobil Katmanı (Client Side) - "Arayüz"

Burada **React Native** ve **Expo** teknolojilerini kullanıyoruz. TypeScript ile yazıldı.

- **Altyapı:** Expo Go (Geliştirme aşamasında kablo/derleme derdi olmadan telefonda test etmek için).
- **Veri Yönetimi (State Management): React Context API (SensorContext).**
 - Veri uygulamanın her yerinden erişilebilir durumda.
- **Haberleşme: Axios.**
 - Uygulama, her 3 saniyede bir (setInterval) Backend'e gidip "Yeni veri var mı?" diye soruyor (Polling yöntemi).
- **Kritik Konfigürasyon:**

- Dinamik IP sorunu olduğu için, SensorContext içinde API adresi manuel olarak (192.168.1.104 gibi) güncelleniyor.
- **UI (Kullanıcı Arayüzü):**
 - Anlık sensör verilerini gösteren kartlar.
 - Sıcaklık verilerini .toFixed(1) ile yuvarlayarak temiz gösterim.

5. Veri Akış Şeması (Data Flow Pipeline)

Sistemin çalışma mantığının özeti şudur:

1. **Algılama:** Sensör ortamı fiziksel olarak ölçer.
2. **İşleme (Edge):** ESP32 bu veriyi okur, 1200 eşiğini geçerse **Buzzer**'ı öttürür.
3. **Paketleme:** ESP32 veriyi JSON formatına çevirir ({ "temp": 24.5, "mq9": 350... }).
4. **İletim:** Wi-Fi üzerinden HTTP POST isteği ile senin bilgisayarının IP'sine (192.168.1.104:8000) atar.
5. **Kaydetme:** Backend bunu alır, Supabase'e yazar.
6. **Görüntüleme:** Mobil uygulama Backend'den son veriyi çeker ve ekrana basar.